

Homework 8 - Discussion

For this Ben Olschar, 108021211678 and Frederik Hüttemann, 108021215247 cooperated with each other. We did implement the code on our own, but agreed to using one version. The discussion and results are made in cooperation.

Task Configuration

For the GPU algorithms, we needed the `BLOCK_SIZE` and `GRID_SIZE` to launch the kernels. The `BLOCK` and `GRID_SIZE` for the cascaded algorithm from HW7 could be copy-pasted as they were already calculated in the last exercise.

To launch the kernel for the HW8 kernel, we need to calculate the number of active block on our GPU. For this the GPU config has been read using `cudaGetDeviceProperties`. First of all, we fixed the number of persistent blocks approximately equal to the maximum number of concurrently active block. Therefore `BLOCK_SIZE = 512`;

The `GRID_SIZE` is the calculated using the following formula, to maximize the number of activate threads:

$$\text{GRID_SIZE} = \text{number of SMs} \cdot \frac{\text{max threads per SM}}{\text{BLOCK_SIZE}} = 40 \cdot \frac{1024}{512} = 80$$

Therefore, we set `GRID_SIZE = 80`.

Task 1: CPU reduction algorithm

Here, a baseline on CPU is implemented used to compute the results. The reduction algorithm simply returns the sum of the array. The CPU is the slowest test because of sequential operations. The code has been copied from the last HW7 to maintain correctness.

Task 2: Harris Cascaded Algorithm with thread-fence (HW8)

The core idea of the optimizations is to calculate the sum for the reduction in shared memory incrementally. With that we are able to merge threads and add their values in a tree-like manner. The optimizations challenge multiple stumbling blocks that can lead to sequential operations and therefore slower calculation. They solve problems with warp-divergence, bank-conflicts, idle threads and reducing work as execution of every for-loop iteration is unnecessary. The following optimizations from the lecture were used:

- **Reduction #5 / #6: Unroll last warp / complete unrolling:** removing unnecessary executions of iterations of the loops and if-statements by using the keyword `volatile`. This gets rid of reduction #3, which has further optimized

#1 and #2. Therefore, reductions #1-#3 are not needed, when using warp unrolling.

- **Reduction #7: Multiple Adds per Thread:** improves loading the data by already performing the first addition. This optimizes the Reduction #4 where only a single addition is performed on loading.

- **__threadfence:** This operation is used to order read/write operations. Using this, we can find the last block of a grid. This last block sums over all block results of its grid.

Task 3: Comparing with GPU reduction using cascaded algorithm (HW7)

The cascaded reduction algorithm was copy-pasted last homework, so we can compare the Harris algorithm. As already stated in the last homework, the `atomicAdd` cascaded algorithm is significantly faster than the sequential CPU algorithm as this makes use of parallelism and tries to avoid blocking usage of `atomicAdd`.

Comparison Because the last week's algorithm is already pretty optimized, no real speedup was expected. However, the Harris algorithm should be a bit faster.

The atomic cascaded algorithm took 0.22 ms on average to reduce the 40 MB float. The implementation of this week's homework is roughly 1.3x faster. This is because we got rid of the `atomicAdd` which can lead to blocking threads as the addition is still performed in sequential manner. Eventhough we used a cascaded algorithm in the last HW7, the additions are still the bottleneck.

For the Harris algorithm, no `atomicAdd` has been used which leads to non-blocking additions within warps. Additionally, we don't have bank conflicts when using the shared memory, which further optimizes the algorithm. Because the HW7 algorithm is already pretty good, the speedup of x1.3 is great, as the number of additions for both algorithms are roughly the same. (Both use a tree-like sequence of operations).